

# Tema 02. Clasificación y Regresión

**Autor: Ismael Sagredo Olivenza**

## 2.0 Base matematica.

### **Derivada Parcial.**

Las derivadas parciales son una extensión natural de las derivadas ordinarias que se aplican a funciones de varias variables. Mientras que las derivadas ordinarias se enfocan en la tasa de cambio de una variable con respecto a otra en una función de una sola variable, las derivadas parciales nos permiten entender cómo cambia una función en relación con cada una de sus variables mientras las demás se mantienen constantes.

Por ejemplo, si tenemos la función:

$$f(x, y) = 2x^2 + xy + 6$$

Hay dos variable X y Y.

Seleccionando una para indicar con respecto a quien de las dos se va a derivar, podemos elegir primero la X (dejando la Y como constante) esto se denota de la siguiente forma:

$$\frac{\partial}{\partial x} = 2x^2 + xy + 6$$

Como se va a derivar con respecto a X, todas las Ys son constantes.

Derivando el resultado sería:

$$\frac{\partial}{\partial x} = 4x + y$$

si deseamos derivas respecto a Y sería:

$$\frac{\partial}{\partial y} = x$$

# Gradiente

El gradiente es equivalente a la derivada de una variable, aplicada a múltiples variables. Se resuelve produciendo un vector con tantas dimensiones como variables haya. En cada dimensión se calculará la derivada parcial con respecto a una de las variables. Indica la dirección de máximo cambio de la función. La magnitud del gradiente es la pendiente de la gráfica en esa dirección. El gradiente se representa con el operador diferencial **nabla**  $\nabla$

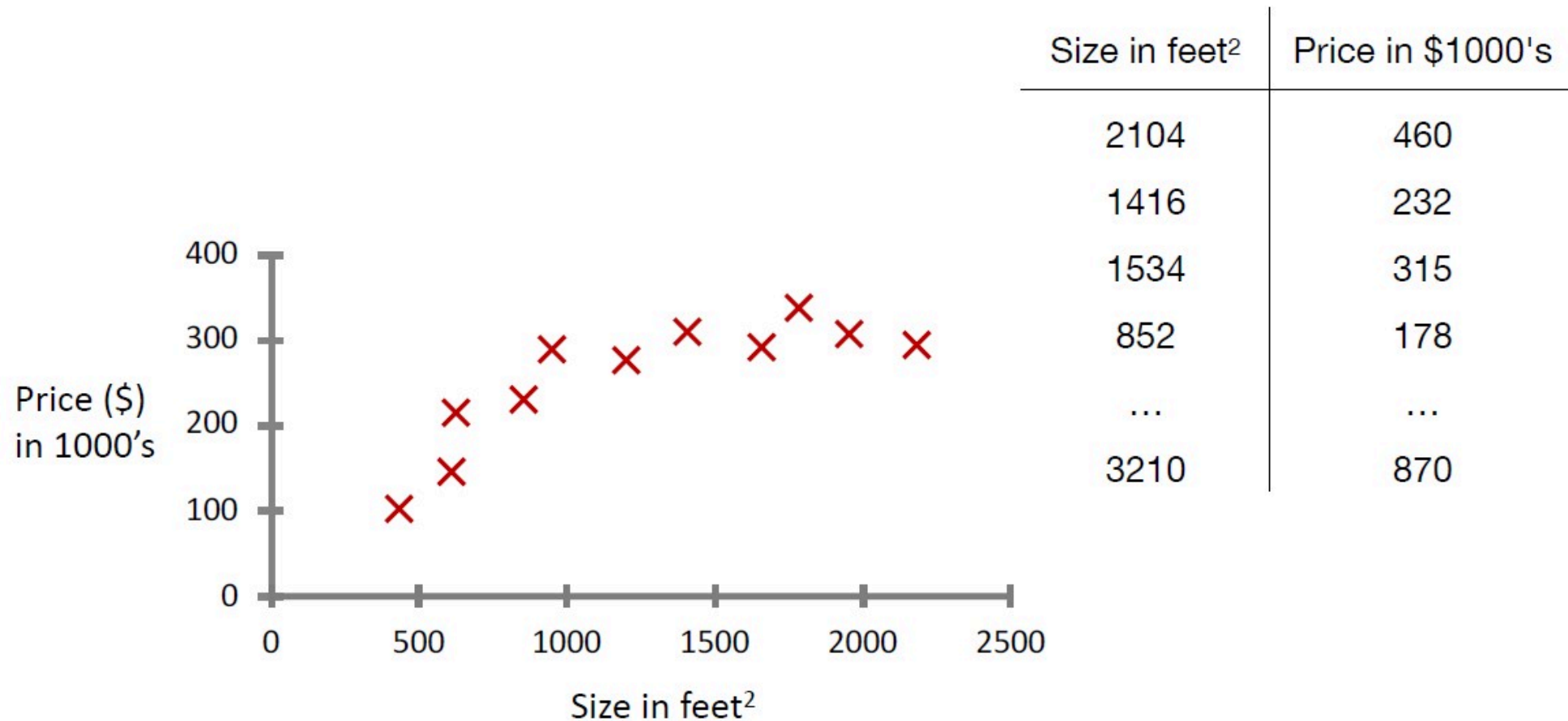
$$f(x, y) = 2x^2 + xy + 6$$

$$\nabla f(x, y) = \left( \frac{\partial}{\partial x}, \frac{\partial}{\partial y} \right) = (4x + y, x)$$

## 2.1 Regresion lineal de una variable

- Es el modelo estadístico más simple.
- Es lineal: asume una relación lineal entre la entrada (X) y la salida (Y)
- Cuando hay una única variable de entrada se denomina **Regresión lineal simple**.
- Cuando hay múltiples variables de entrada se denomina **Regresión lineal múltiple o multivariable**.
- Se suele usar como modelo base para comparar otros algoritmos.

# House sizes and prices



## 2.1.1 Terminología

- Los datos de entrenamiento (**Training Set**) se utilizan para entrenar el modelo
- $X$  : entrada (también denominada característica)
- $Y$  : salida (también llamada objetivo)
- Ejemplo: una de las filas de la tabla que vimos en la imagen anterior.  $(X,Y)$
- Los datos de validación (**Validation set**) sirven para probar el aprendizaje.



## 2.1.2 Modelo de regresión lineal

- Es la ecuación

$$Y = w * x + b \Rightarrow f(w, b) = w * x + b$$

- ¿Cómo calculamos w y b?
- Hay diferentes técnicas para hacerlo:
  - Por mínimos cuadrados (esta es la que os enseñaron en el insti)
  - Descenso de gradiente
  - Regularización

## 2.1.3 Función de coste (Cost function)

- Es una expresión matemática que permite a los algoritmos de machine learning analizar **como de bien se ajusta el modelo a los datos de entrenamiento**
- La salida de la función de coste representa la desviación del modelo con respecto a los datos reales.
- Se pueden usar multitud de funciones de coste, pero una de las más usadas es el **error cuadrático medio (mean square error (MSE))**

## 2.1.3.1 Mean square error (MSE)

$$MSE(Y, Y') = \frac{1}{m} \cdot \sum_{i=1}^m (y_i - y'_i)^2$$

- Distancia entre  $y'_i$  (estimada  $\Rightarrow f_{w,b}(x)$  en regresión lineal) y el valor real  $y_i$  elevando al cuadrado (mantiene error positivo)
- Se suman todas las desviaciones y se calcula la media.
- Como veremos más adelante, a veces se divide por 2 para facilitar el cálculo de la derivada de la función de costes.

$$MSE(Y, Y') = \frac{1}{2m} \cdot \sum_{i=1}^m (y_i - y'_i)^2$$

## Explicación de la formula:

Se generan para cada ejemplo la predicción basandonos en este caso en la recta de regresión

$$y' = m \cdot x + b$$

X es la entrada, y M y b es lo que hay que calcular para que el error sea el minimo.

Esto obtiene una lista o vector de  $y'$ . Debemos tener otro vector de y reales etiquetados en el dataset. Asi que restamos el valor de  $y' - y$  y lo elevamos al cuadrado para quitarle el signo. Despues sumamos todo.

## 2.1.3.2Cuál es el objetivo de la regresión lineal?.

- Minimizar la función de coste todo lo que se pueda.

### Veamos un ejemplo

Llamemos a la función de coste **J**. Por tanto  $f_{w,b}(x)$  será  $J(w, b)$  (donde  $w$  y  $b$  son los parámetros que queremos calcular).

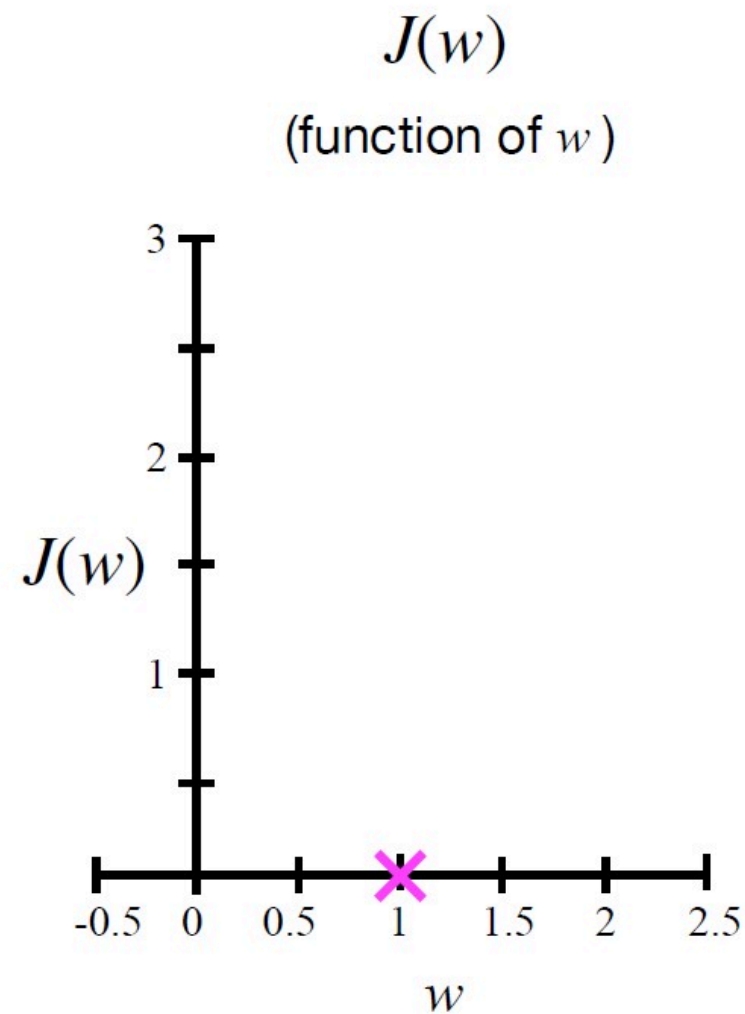
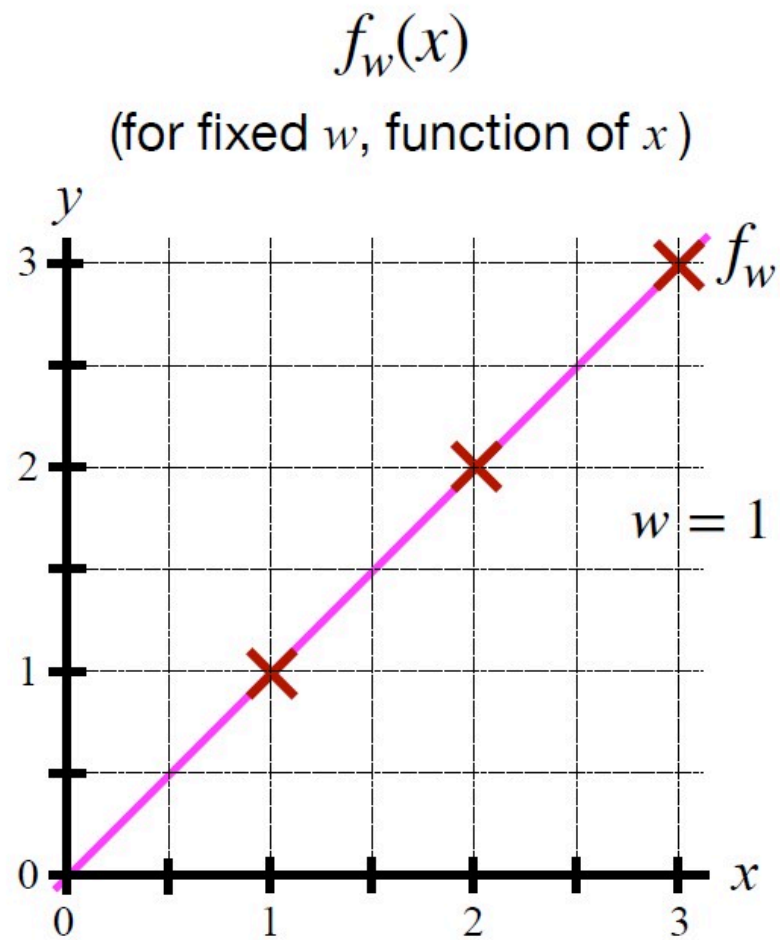
Comenzemos por simplificar el modelo eliminando el parámetro  $b$ .

$$J(w) = w * x$$

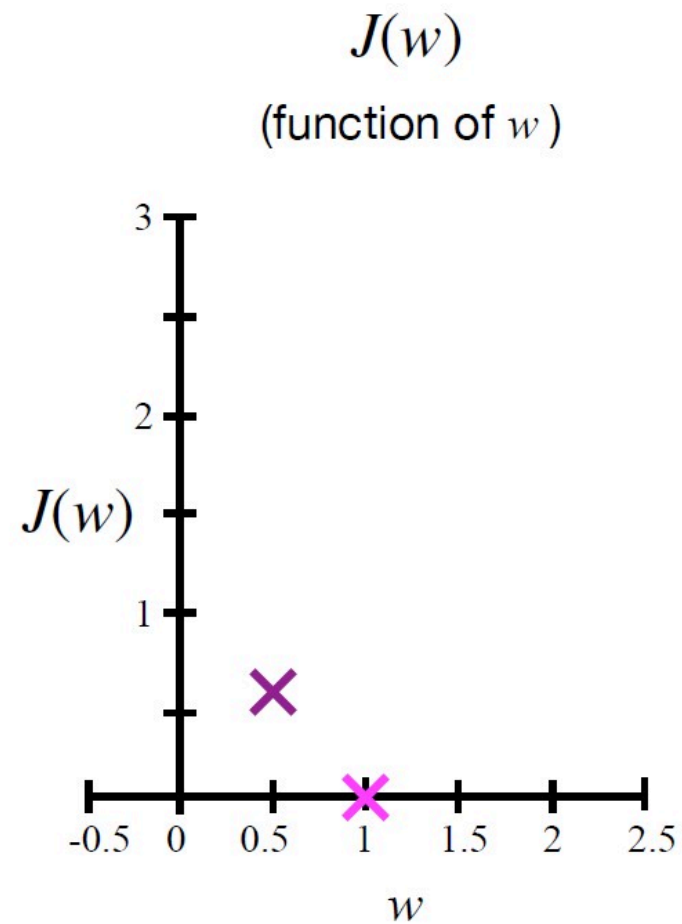
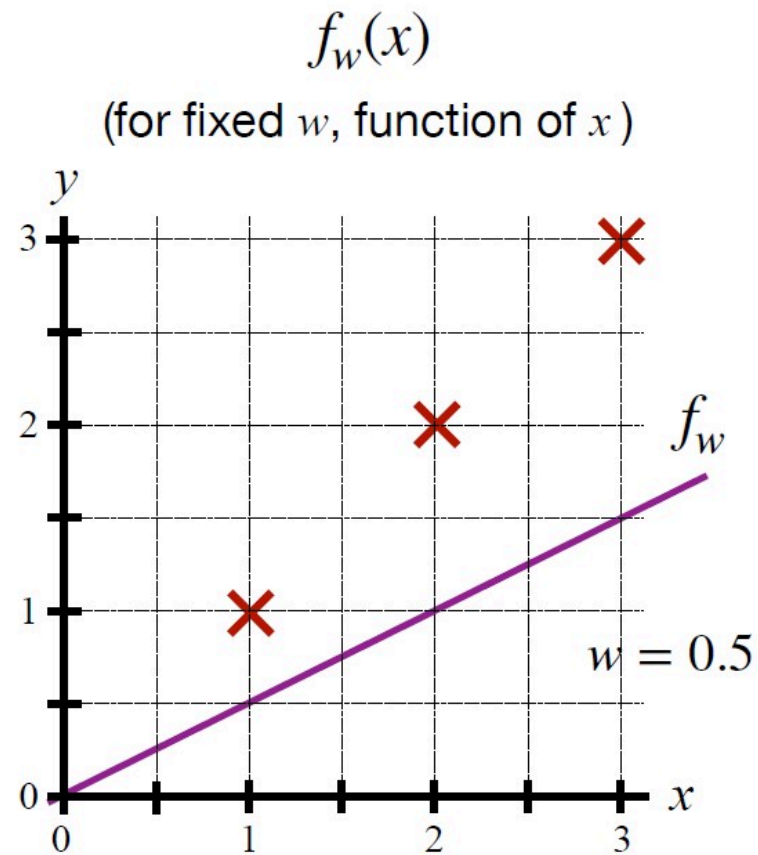
- Asumamos estos datos ficticios como datos del problema a aprender:

| <b>X</b> | <b>Y</b> |
|----------|----------|
| 1        | <b>1</b> |
| 2        | <b>2</b> |
| 3        | <b>3</b> |

Como no sabemos cuanto vale  $W$ , comencemos con  $W = 1$ , luego con  $W = 0.5$  y así sucesivamente para ir viendo el error o coste ( $J$ ) cometido por la recta en cada caso y dibujaremos el error en una grafica para ir viendo como evoluciona.

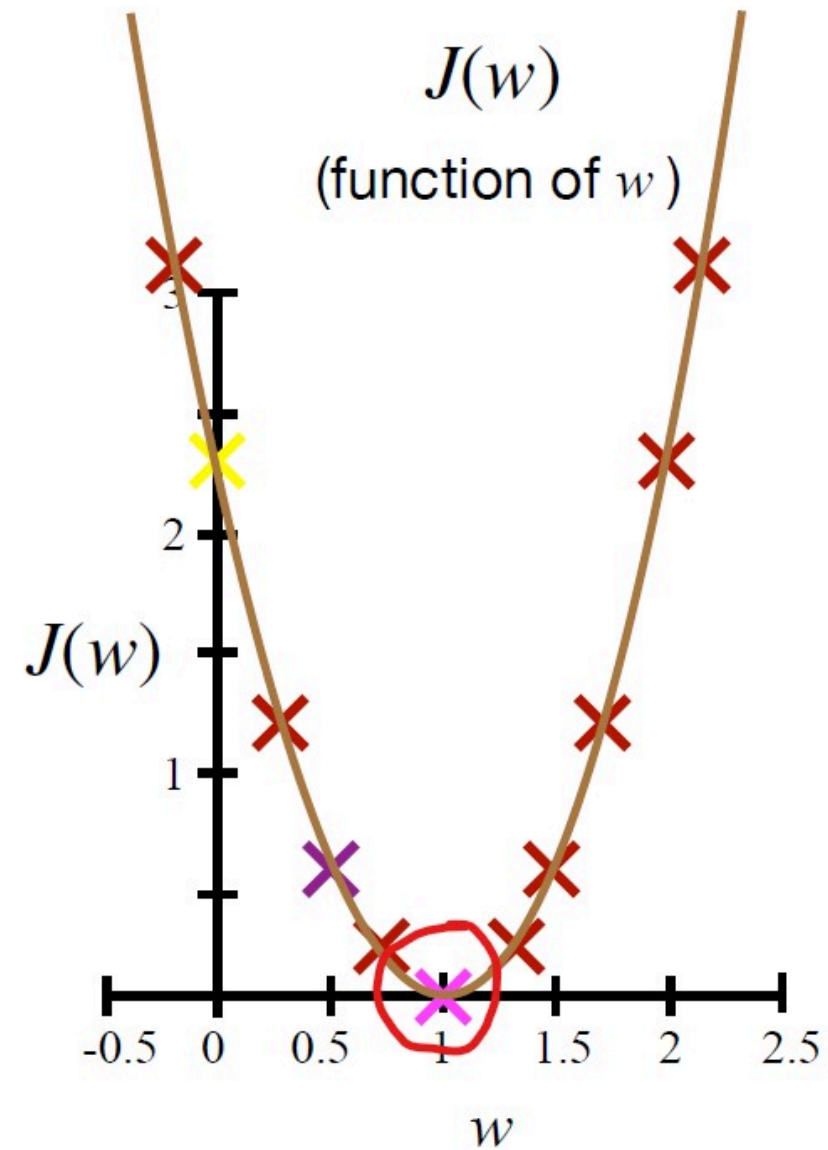
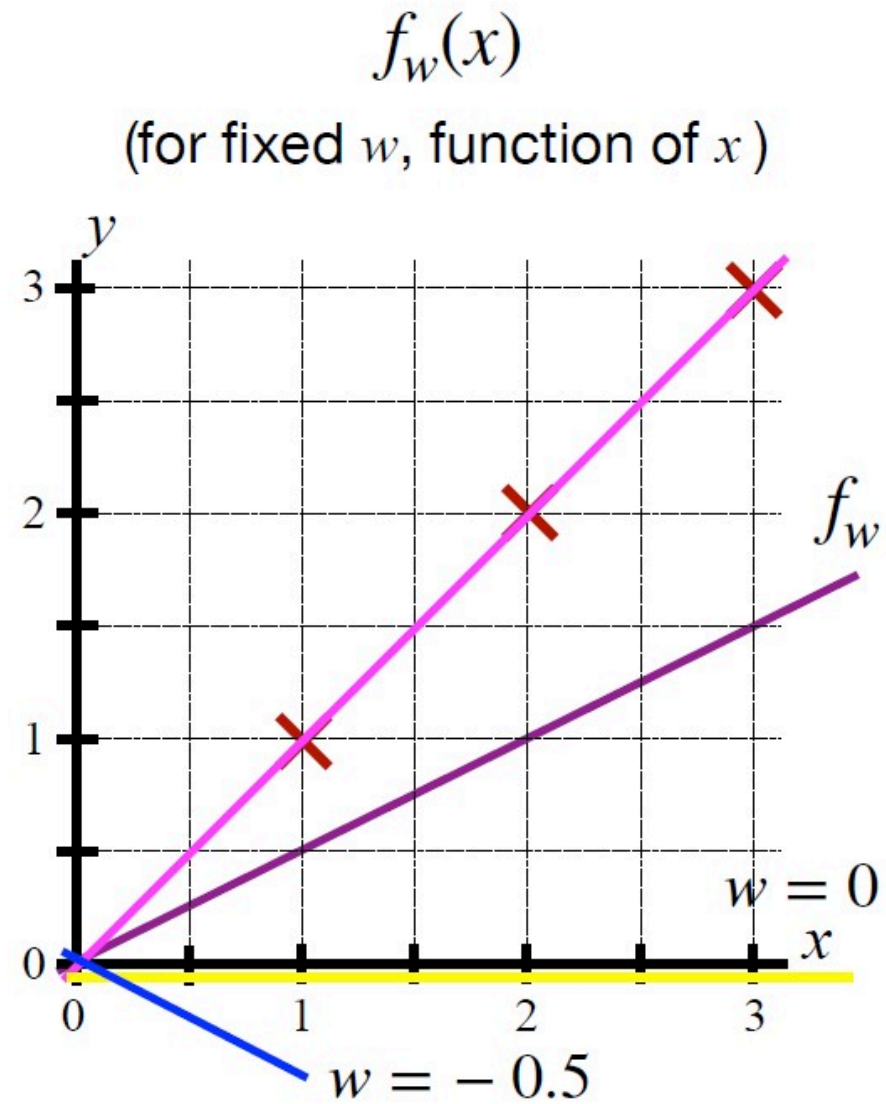


$$J(1) = \frac{1}{2m} \sum_{i=1}^m (f_w(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} - y^{(i)})^2 = \frac{1}{2m} (0^2 + 0^2 + 0^2) = 0$$



$$J(0.5) = \frac{1}{2m} \cdot \sum_{i=1}^m (wx_i - y'_i)^2 = \frac{1}{2m} (0.5^2 + 1^2 + 1.5^2) = \frac{1}{6} (0.25 + 1 + 2.25) = 0.58$$





## 2.2 Descenso de gradiente

- Podemos calcular  $W$  usando este algoritmo de optimización.
- El objetivo es minimizar  $J(w, b)$
- Cambiar  $w$  y  $b$  hasta conseguir un valor lo más cercano posible al mínimo usando la derivada del error (o función de coste):

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

- $\alpha$  es la velocidad con la que cambiamos  $w$  (factor de aprendizaje o learning rate)

El descenso de gradiente para los dos parámetros sería:

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

Detalles de implementación: ojo no cambiar los valores  $w$  y  $b$  en los cálculos ya que si cambiamos  $W$  y después calculamos  $b$ , el cálculo de  $b$  sería incorrecto, mejor hacerlo con variables locales.

# Gradient descent algorithm

Repeat until convergence

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$
$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

Code assignment

$$a = c$$

$$a = a + 1$$

Math

$$a = c$$

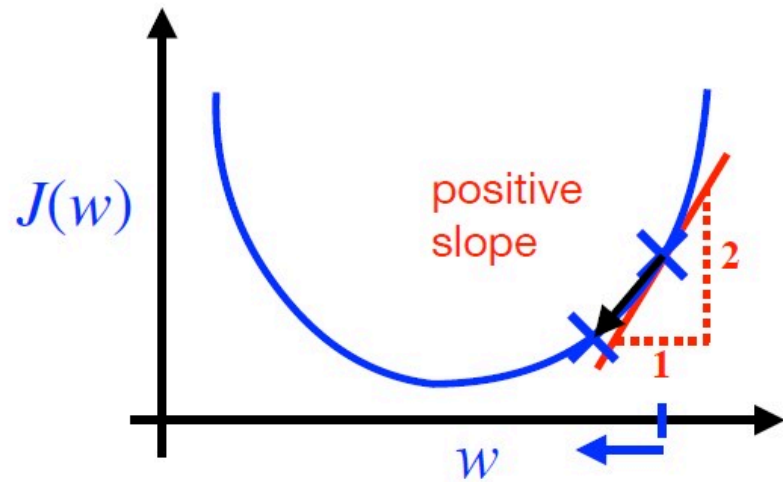
$$a \neq a + 1$$

Correct: simultaneous update

$$tmp\_w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$
$$tmp\_b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$
$$w = tmp\_w$$
$$b = tmp\_b$$

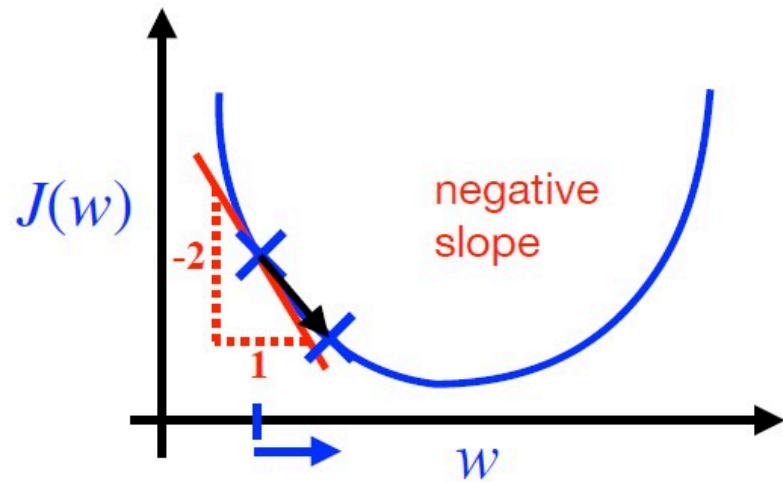
Incorrect

$$tmp\_w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$
$$w = tmp\_w$$
$$tmp\_b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$
$$b = tmp\_b$$



$$w = w - \alpha \frac{d}{dw} J(w)$$

$$w = w - \alpha \cdot (\text{positive number})$$



$$\frac{d}{dw} J(w) < 0$$

$$w = w - \alpha \cdot (\text{negative number})$$

## 2.2.1 Descenso de gradiente en regresión lineal

- Recordemos la función de coste es:

$$J(w, b) = \frac{1}{2m} \cdot \sum_{i=1}^m (f_{w,b}(x_i) - y_i)^2$$

$$w = w - \alpha \frac{\partial J(w, b)}{\partial w}$$

$$b = b - \alpha \frac{\partial J(w, b)}{\partial b}$$

## 2.2.1.1 Aplicando la derivada

Recuerda que:  $\partial x^2 = 2x$

$$\frac{\partial J(w, b)}{\partial w} = \frac{1}{2m} \cdot \sum_{i=1}^m (wx_i + b - y_i)^2 = \frac{1}{2m} \cdot \sum_{i=1}^m ((y_i' - y_i) \cdot 2x_i) = \frac{1}{m} \sum_{i=1}^m (y_i' - y_i) * x_i$$

Recuerda que:  $\partial x = 1$

$$\frac{\partial J(w, b)}{\partial b} = \frac{1}{2m} \cdot \sum_{i=1}^m (wx_i + b - y_i)^2 = \frac{1}{2m} \cdot \sum_{i=1}^m ((y_i' - y_i) \cdot 2) = \frac{1}{m} \sum_{i=1}^m (y_i' - y_i)$$

## 2.2.2.2 Algoritmo de descenso de gradiente

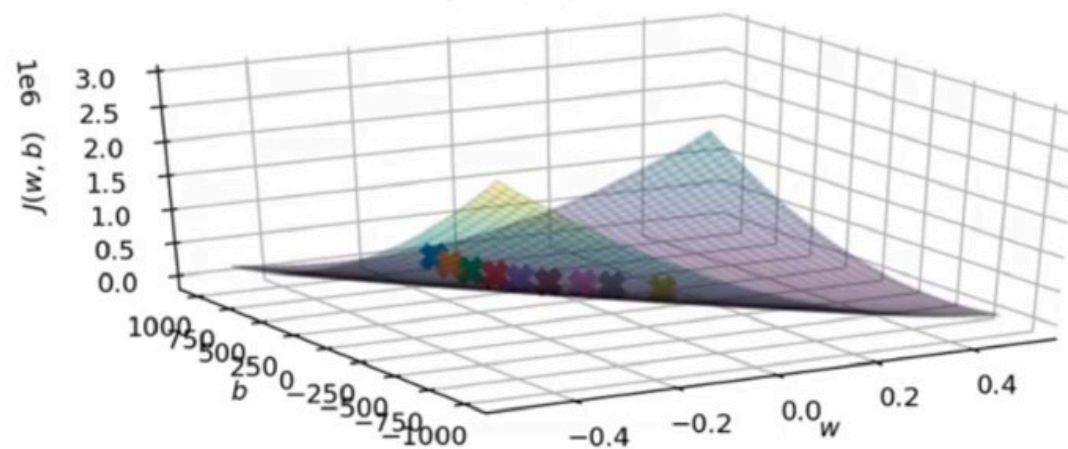
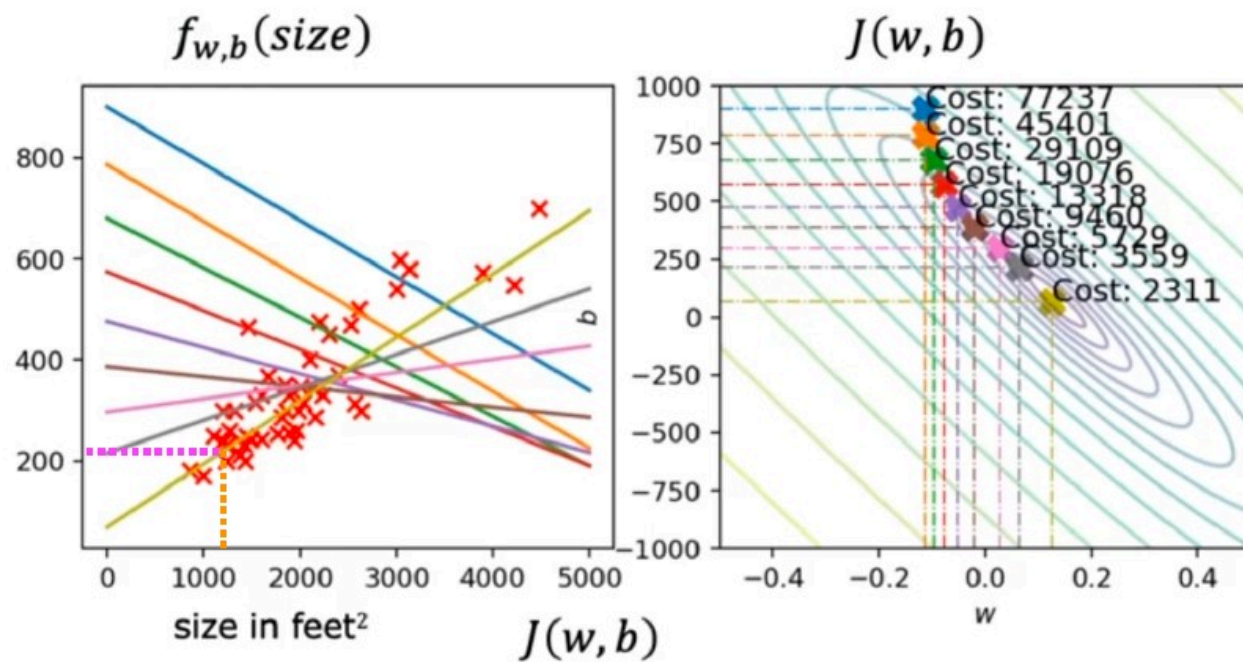
Repetir hasta la convergencia.

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^m (y_i' - y_i) * x_i$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (y_i' - y_i)$$



price in  
\$1000's



## 2.2.2.2 Bach

En función del número de ejemplos de entrenamiento considerados en la actualización de los parámetros tenemos 3:

- **Descenso Gradiente por Lotes:** Los parámetros se actualizan tras calcular el gradiente del error con respecto **a un lote**.
- **Descenso de gradiente estocástico:** Los parámetros se actualizan después de calcular el gradiente del error con respecto a un **único ejemplo de entrenamiento**.
- **Descenso Gradiente Mini-Lotes:** Los parámetros se actualizan después de calcular el gradiente del error con respecto a **un subconjunto del conjunto de entrenamiento**.

## 2.2.2.2 Algoritmo Mini-Bach

- **Forward pass** sobre ejemplos seleccionados del lote.
  - Realizar la predicción sobre el minilote  
Calcular el error de predicción ( $J$ )
- **Backward pass** calcular de descenso de gradiente
- Actualizar parámetros